

Vanderbilt University
Department of Electrical and Computer Engineering

Real-Time Audio Spectrum Analyzer on FPGA

ECE 4377 – FPGA Design Final Project

Prepared by:

Battulga Purev
Xavier Onwu
Tausif Samin

Target Hardware:

Altera Cyclone IV E (DE2-115)

May 9, 2026

1 Introduction

Real-time audio processing requires strict deterministic timing and high-throughput parallel computation capabilities. FPGAs are highly suited for Digital Signal Processing (DSP) operations like the Fast Fourier Transform (FFT) due to their inherent parallelism and ability to process streaming data without instruction-fetch latency. The objective of this project is to create a fully integrated Audio Spectrum Analyzer that responds instantaneously to ambient audio and visualizes frequency content seamlessly.

The project emphasizes hardware architecture design, state-machine-driven data flow, I2C/I2S communication protocols, and complex mathematical realization in VHDL without the reliance on a soft-core processor (such as NIOS II). By managing all peripheral interfacing and DSP algorithms through dedicated custom logic, the system minimizes latency and maximizes throughput.

2 System Architecture

The system architecture follows a linear, heavily pipelined data flow from the analog front-end to the digital visual outputs, operating across multiple distinct clock domains.

- **Input Stage:** The physical microphone feeds the WM8731 Audio Codec, which handles analog-to-digital conversion.
- **Clocking & Configuration:** A hardware phase-locked loop (PLL) synthesizes a 12.288 MHz master audio clock and a 25.175 MHz VGA pixel clock from the board's native 50 MHz oscillator. An I2C master orchestrates the register initialization of the WM8731 chip.
- **Serialization & CDC:** The I2S capture modules utilize 2-stage synchronizers to safely cross clock domains, aggregating the serial bit-stream into 16-bit parallel samples and buffering them into 256-sample frames.
- **Computation Engine:** An 8-stage Radix-2 FFT core converts the 256 time-domain samples into frequency bins utilizing a pre-calculated Twiddle Factor ROM.
- **Output Stage:** A Peak Tracker analyzes the magnitudes, routing data to a custom VGA raster generator, binary-to-BCD seven-segment displays, and a dynamically decaying LED VU meter.

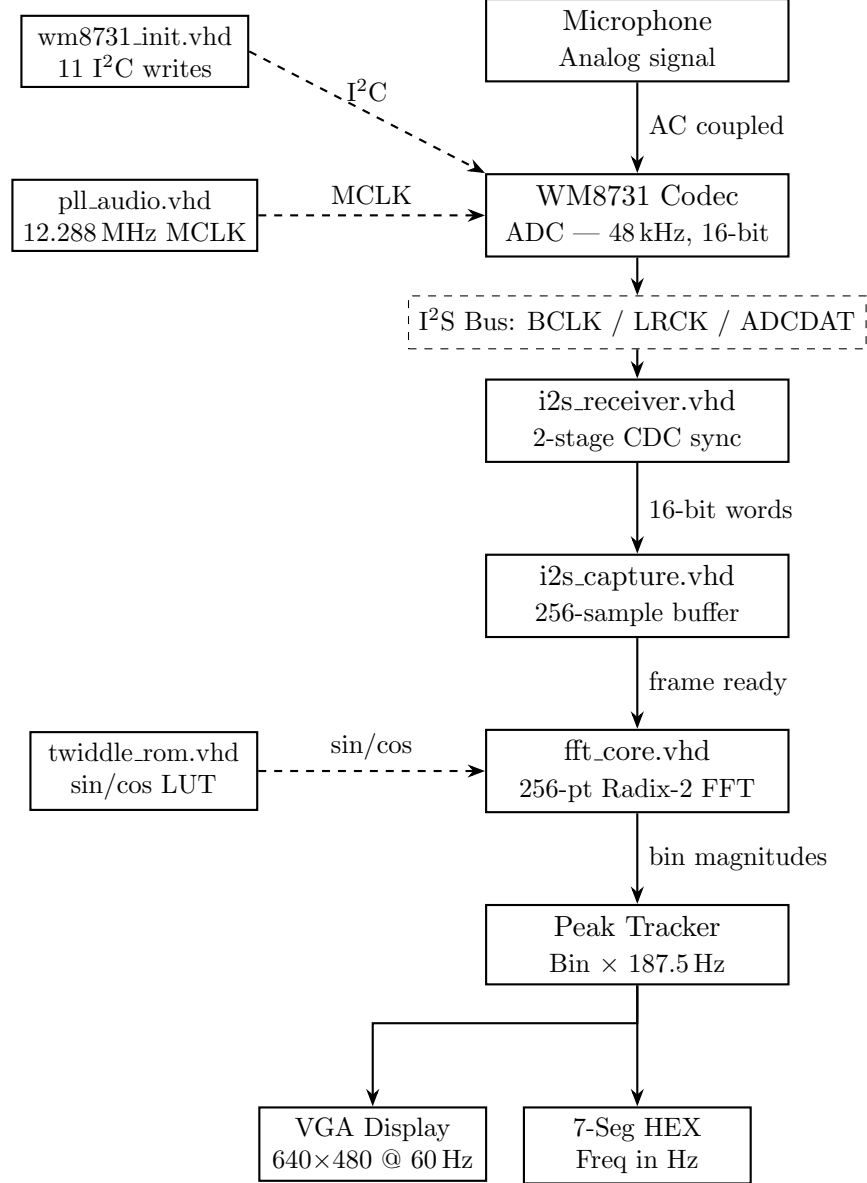


Figure 1: Top-level hardware architecture detailing the audio pipeline from the WM8731 codec to the VGA and LED visualizers.

3 Hardware Implementation Details

3.1 Audio Codec Hardware Interface

3.1.1 Codec Overview and Clock Architecture

Device: Wolfson WM8731 is a low-power stereo audio codec with integrated ADC/DAC.

Role: ADC path only, digitizes analog microphone input and streams to FPGA over I2S.

Clock requirement: $MCLK = 256 \times f_s$

$$f_s = 256 \times 48,000 = \mathbf{12.288\text{ MHz}}$$

Signal	Source	Value
Board oscillator	DE2-115	50 MHz
MCLK (synthesized)	<code>p11_audio.vhd</code>	12.288 MHz
Sample rate f_s	WM8731 ADC	48 kHz
BCLK	FPGA	3.072 MHz ($64 \times f_s$)
LRCK	FPGA	48 kHz

Table 1: Audio clock hierarchy

3.1.2 Codec Initialization (`wm8731_init.vhd`)

The WM8731 powers up with all circuitry disabled and all paths muted by default. This prevents audible pop noise during power supply ramp-up. The FPGA must write 11 control registers over I2C before the codec produces any output.

I2C transaction parameters:

- Clock rate: 400 kHz (fast mode)
- Device address: 0x1A (set by CSB pin logic level)
- Word format: 16 bits — [15:9] register address, [8:0] data
- Each write: START $\rightarrow$ address+W $\rightarrow$ ACK $\rightarrow$ data[15:8] $\rightarrow$ ACK $\rightarrow$ data[7:0] $\rightarrow$ ACK $\rightarrow$ STOP

Initialization sequence — 11 register writes in order:

Order	Register	Value	Purpose
1	R15 — Reset	0x000	Force codec to factory defaults. Always first.
2	R0 — Left line in	MUTE	Mute unused line input
3	R1 — Right line in	MUTE	Mute unused line input
4	R2 — Left HP out	0 dB	Set defined headphone output state
5	R3 — Right HP out	0 dB	Set defined headphone output state
6	R4 — Analog path	INSEL=MIC, ...	Route mic to ADC, enable +20 dB boost.
7	R5 — Digital path	ADCHPD=0, DACMU=1	Enable DC-offset HPF, soft-mute DAC
8	R6 — Power down	0x000	Power on mic, ADC, outputs.
9	R7 — Interface format	I2S, 16-bit, slave	Set I2S framing, 16-bit word, master clock
10	R8 — Sampling	48 kHz, 256fs	Configure filter coefficients for 48 kHz
11	R9 — Active	0x001	Release audio interface from tristate.

Table 2: WM8731 initialization register sequence

Key register decisions:

- **R4 MICBOOST (+20 dB):** The DE2-115 onboard electret mic has low output sensitivity. Without this boost the ADC input is dominated by noise floor rather than the acoustic signal.
- **R5 HPF enable:** The on-chip digital HPF removes DC offset using $H(z) = \frac{1-z^{-1}}{1-0.9995z^{-1}}$, with $f_{-3dB} \approx 3.7$ Hz. Without this, residual DC would produce a large spurious spike in FFT bin 0, corrupting peak detection.
- **R9 last:** Writing ACTIVE=1 before R7/R8 are configured risks the interface coming up in an undefined format state. The WM8731 datasheet mandates R9 as the final write.

3.1.3 I2S Data Capture and Clock Domain Crossing

I2S bus signals:

I2S framing (WM8731 I2S mode):

Signal	Frequency	Direction	Function
BCLK	3.072 MHz	FPGA → codec	Bit clock — one pulse per serial bit
LRCK	48 kHz	FPGA → codec	Low = left channel, high = right channel
ADCDAT	—	Codec → FPGA	Serial audio data, MSB first

Table 3: I2S bus signal summary

- MSB appears **one BCLK after** the LRCK transition — this is the defining I2S offset vs. left-justified format
- Data is valid and stable before the BCLK falling edge — capture logic samples on falling edge
- Only left channel (LRCK = 0) is captured — microphone input is mono

3.1.4 Sample Accumulation and FFT Handoff (`i2s_capture.vhd`)

Per-sample pipeline:

1. `data_valid` strobe asserted by `i2s_receiver` once per left-channel word (every 20.8 μ s)
2. Sample written to current address in 256-word RAM buffer
3. Write pointer incremented
4. At pointer = 255: frame complete

Frame handoff:

- `frame_done` pulse asserted — triggers FFT execution
- Ping-pong buffer toggled — incoming samples immediately fill alternate buffer with zero sample loss
- LEDG3 pulses at 187.5 Hz as a visible heartbeat confirming continuous acquisition

Why 256 samples:

- Frame size = FFT size: one audio frame maps to exactly one transform — no overlap, zero-padding, or sample discard required

Parameter	Value
Frame size N	256 samples
Sample rate f_s	48,000 Hz
Frame duration	$N/f_s = 5.33$ ms
FFT transform size	256 points
Frequency resolution	$\Delta f = f_s/N = 187.5$ Hz/bin
Useful bins	128 (DC to Nyquist)

Table 4: Frame and FFT parameters

- $\Delta f \approx 188$ Hz/bin determines the frequency precision of peak detection and 7-segment display output

3.2 Fast Fourier Transform (FFT) Core

The computational core of the project consists of two main hardware description files, which are `fft_core.vhd` and `twiddle_rom.vhd`. The `fft_core.vhd` is the main brain of the process. It loads the converted 16 bit audio data from the `i2c_master.vhd` module to a local M9K Block RAM on the FPGA. Once 256 data points are collected, it performs a custom 256-point Radix-2 discrete fourier transformation. To avoid introducing complexity to the main module by computing expensive trigonometric calculations, the complex twiddle factors required for the butterfly mixing stages are stored in a separate Read-Only Memory module `twiddle_rom.vhd`. This module serves as a hardware look-up table, providing the pre-calculated sine and cosine coefficients necessary for phase rotation.

Due to potential overflowing in the butterfly addition, the FFT engine was designed such that it can manage the overflow. Although input samples are 16-bit signed integers, the internal RAM buffers (`ram_re`, `ram_im`) are scaled to 32-bit signed vectors to prevent overflow during intermediate stages. The operation proceeds as follows:

1. **Load Phase:** The 256 time-domain samples are transferred into the real-part memory, while internal imaginary memory is zeroed.
2. **Butterfly Execution:** Over 8 stages, the finite state machine computes the symmetric pairs of data. Complex multiplication utilizing the fetched Twiddle vectors is executed

against the lower matrix, and the updated elements are written back to memory.

$$O_A = \frac{A + (B \cdot W)}{2}$$

$$O_B = \frac{A - (B \cdot W)}{2}$$

Where $W = \cos(\theta) - j \sin(\theta)$, pre-computed by *twiddle_rom.vhd*.

3. **Magnitude Estimation:** The output frequency domain data exists as complex numbers. The absolute scalar magnitude of the result is calculated.

$$\text{Mag} \approx \text{MAX}(|\text{Re}|, |\text{Im}|) + \frac{\text{MIN}(|\text{Re}|, |\text{Im}|)}{4} \quad (1)$$

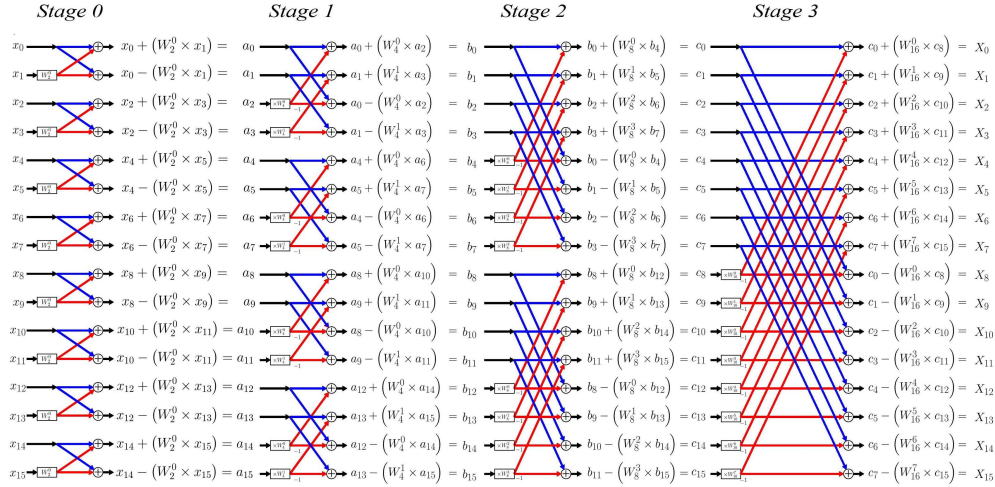


Figure 2: A single Radix-2 butterfly computational unit utilized within the 8-stage FFT pipeline.

3.3 Peak Tracking Logic

Implemented inside the top-level entity (*spectrum_analyzer_top.sv*), the peak detection circuitry evaluates the FFT outputs in real time. For every new 256-point frame, the system scans through the outgoing magnitudes. It intentionally bypasses Bin 0 (the DC offset) to suppress static baseline noise. The highest amplitude and its corresponding frequency bin index are latched.

Given a sampling frequency (F_s) of 48 kHz and an FFT length (N) of 256, the frequency

resolution per bin is defined as:

$$\Delta f = \frac{F_s}{N} = \frac{48000}{256} = 187.5 \text{ Hz} \quad (2)$$

The dominant frequency is thus dynamically evaluated as $\text{Bin}_{\text{Peak}} \times 187.5 \text{ Hz}$.

4 Output and Display Interfaces

4.1 VGA Display Controller

This subsystem in our design translates the frequency arrays from the FFT core into real-time visual parameters via a VGA display. Our display takes advantage of an Intel IP core PLL to turn our input 50 MHz clock signal into the 25.175 MHz signal required for VGA output at a resolution of 640×480 at 60 Hz.

4.1.1 FFT Magnitude to Bar Graph

The FFT core calculates a 16-bit magnitude for 128 discrete frequency bins per audio frame. Once processed, this array of magnitudes is pushed sequentially to the `vga_spectrum` module for real-time visualization. The operations for the bar graph procede as follows:

- **Memory Buffer:** A 128×16-bit dual-port RAM block is instantiated to store the incoming frequency bins. The FFT core writes the 128 magnitudes into this buffer, effectively isolating the high-speed audio processing domain from the display timing domain.
- **Drawing the Plot:** The rendering logic first constructs the static UI elements based on the current pixel coordinates. The X -axis is drawn horizontally at $Y = 320$ (spanning $X = 100$ to 540), and the Y -axis is drawn vertically at $X = 108$ (spanning $Y = 20$ to 320). Following the bounding axes, dynamic tick marks and standard-font numerical labels are rendered at specific coordinate boundaries to establish a precise measurement scale.
- **Drawing the Bars:** To draw bars the hardware must first determine if the current pixel position is within the bar graph grid. After confirmation, the position of the grid

is tracked in sets of 4, and a check is performed to ensure that a pixel can only be altered if it is in the first two of a set of 4. This ensures that there is always a 2 pixel gap between each bar, and 4 pixels belong to each frequency bin. The address for the frequency bins is directly tied to the pixel position in such a way that it updates after the pixel position has moved horizontally 4 times ensuring a proper sync of data. The hardware grabs the value from the current frequency bin based on that address and shifts the corresponding value 5 bits to the right to allow it to fit in our VGA display. Next, the value is clamped to a maximum of 300. This is due to 300 pixel limit ($320-20=300$). Because VGA monitors draw top-to-bottom ($Y = 0$ at the top), the logic applies a Y -axis inversion: a pixel is illuminated if its upward distance from the bottom axis ($320 - Y$) is less than the calculated bar height. Due to the mathematical dependence of the bin address on the pixel count of the screen, frequencies in bins 106 to 127 that are stored in memory are never actually graphed because the address loops back to 0.

4.1.2 FFT Magnitude Waterfall Heatmap

The waterfall spectrogram continuously renders a scrolling history of frequency amplitudes, using a variety of colors to indicate intensity.

The operations for generating the heatmap proceed as follows:

- **Memory Buffer:** Unlike the bar graph, because a history is present, it would take an excessive amount of BRAM to store 128 frequency bins at their full 16-bit resolution. To account for this the raw magnitude received from the FFT core undergoes a non-linear quantization via a custom `compress8` function. This algorithm applies piecewise bit-shifting to logarithmically compress the wide dynamic range into an 8-bit value, halving the memory footprint while preserving both low-amplitude signal details and high-amplitude peaks. The resulting data is routed to a 16,384 $\times$ 8-bit dual-port RAM. This ram is explicitly defined as M9k BRAM to prevent using excessive logic cells.
- **Efficient Buffering:** To prevent the process of having to rewrite the data at every memory addresses each and everytime data is written to the BRAM, the hardware changes the frame of reference it is using for painting the screen. Each time 128 frequency bins are received for a given audio frame, the memory address corresponding to the top pixel layer is incremented, meaning the previous top memory address is shifted down one. This method saves a lot of computing time and power. This is much

more time and resource efficient compared to keeping a constant memory address corresponding to the top pixel row of the heatmap, forcing the hardware to shift all data in subsequent memory addresses lower.

- **Spacing:** Similar to the linear plot, the horizontal (X -axis) spatial mapping divides the active coordinate by 4 to maintain visual alignment with the 2-pixel wide frequency bars above it. The rendering window for the spectrogram is strictly bounded to the lower quadrant of the screen, actively rendering when the physical Y -coordinate falls between rows 340 and 467.
- **Coloring:** To convert the 8-bit scalar magnitude into a 24-bit RGB pixel, the architecture bypasses the need for a memory-intensive Color Look-Up Table (CLUT) ROM. Instead, the rendering engine synthesizes a thermal colormap on the fly using pure combinational logic. The 8-bit value is evaluated through a series of if statements on the magnitude. Low intensities isolate the blue channel to represent the noise floor, mid-range values ramp aggressively into green, and extreme absolute amplitudes saturate into red and pure white.

4.2 Seven-Segment Displays

We included a seven-segment display and volume meter early in the design process to aid debugging and tune for accuracy. The hardware incorporates a binary-to-BCD conversion system using the double-dabble method to generate base-10 digits for the current frequency output of the FFT core. The peak frequency (in Hz) is output onto the HEX7-HEX4 displays. The raw absolute magnitude peaks are presented in hexadecimal across HEX3-HEX0.

5 Conclusion

This project successfully encapsulates highly advanced DSP techniques within pure digital logic elements on the Cyclone IV E FPGA. By orchestrating everything from asynchronous I2S clock-domain crossings and I2C initialization to 32-bit internal FFT bit-growth management without external processing libraries or microcontrollers, the architecture exemplifies the high-bandwidth, massively parallel computing strengths unique to FPGAs. The resulting Audio Spectrum Analyzer is a reliable, visually responsive tool with strictly bounded, minimal hardware latency.